Document Technique – Architecture ERP & POS Nouvelle Génération

1. Vision Technique Générale

L'objectif est de traduire la vision fonctionnelle (DSF) en une **architecture technique claire, modulaire et évolutive**, adaptée :

- à un **contexte offline-first**
- à un **ERP/POS multi-sites**
- à une **montée en charge progressive (SaaS)**

Stack retenue (et pourquoi)

- **Frontend Web** : Next.js (Back-office, B2B, Admin)
- **POS App Mobile**: React Native + SQLite (rapidité, offline natif)
- **Desktop Admin Local** : Tauri (Rust) + SQLite
- **Backend Central** : NestJS (DDD, modularité, sécurité)
- **Micro-services spécifiques** : FastAPI (Python) pour calculs, sync, IA plus tard
- **Messaging** : Kafka (asynchrone, sync offline, événements métiers)
- **DB Cloud** : PostgreSQL (Neon)
- **Infra** : Docker + Kubernetes

---

2. Architecture Globale (Découpage Macro)

```
[ POS Flutter ]        [ Tauri Desktop ]
       │                       │
       │  (Offline SQLite)     │
       └────── Sync Engine ────┘
                  │
              Kafka Events
```

```
                    │
          ┌──────── NestJS API ────────┐
          │   Auth / RBAC / Core       │
          │   Modules Métier           │
          └────────────┬───────────────┘
                       │
              PostgreSQL (Neon)
```

Principe clé : **tout est événement métier**, jamais du simple CRUD.

---

3. Module Cœur – Catalogue & Produits

Objectif

Créer un **modèle produit polymorphe** capable de supporter 4 comportements sans casser le reste du système.

Découpage Technique

Backend (NestJS)

- Module `catalog`
  - `Product`
  - `ProductVariant`
  - `Batch` (lot)
  - `Serial`

Base de données (PostgreSQL)

- `products` (produit parent)
- `variants` (SKU)
- `batches` (périssables)
- `serials` (IMEI, SN)

Stratégie clé : Type Strategy

Chaque produit possède :

```
product_type: SIMPLE | VARIANT | SERIALIZED | PERISHABLE
```

Et le comportement dépend du type.

Algorithme (exemple vente)

1. Scanner SKU
2. Charger product_type
3. Switch :
   - SIMPLE → décrémenter qty
   - VARIANT → décrémenter variante
   - SERIALIZED → exiger IMEI
   - PERISHABLE → FEFO (lot le plus proche)

Pourquoi ?

➡️ Évite 4 systèmes différents, **un seul moteur de vente**.

---

4. Module Achats & Import (Landed Cost)

Objectif

Calculer le **coût réel**, pas juste le prix fournisseur.

Services impliqués

- NestJS : commandes, réceptions
- FastAPI : calculs financiers (précision, testabilité)

Workflow

1. Création PO (multi-devise)
2. Taux de change figé
3. Ajout frais globaux
4. Réception marchandises
5. Ventilation automatique

Algorithme Landed Cost

```
for article in reception:
  part = article.qty / total_qty
  article.cost = purchase_price + (frais_total * part)
```

Puis recalcul du **CUMP**.

Pourquoi FastAPI ici ?

➡️ Calcul pur, facilement isolable, testable, scalable.

---

5. Module Stock & Logistique (WMS)

Principe

Le stock n'est **jamais modifié directement**.

Tout passe par des événements

- STOCK_IN
- STOCK_OUT
- TRANSFER_OUT
- TRANSFER_IN
- ADJUSTMENT

Architecture

- Table `stock_movements`
- Vue matérialisée `current_stock`

Algorithme transfert inter-sites

1. Boutique crée demande
2. Entrepôt valide
3. Sortie stock (event)
4. Réception boutique (event)

Pourquoi ?

➡️ Audit parfait + reconstruction possible.

---

6. Module POS (Flutter Offline-First)

Pourquoi Flutter

- UI rapide
- Matériel POS compatible
- SQLite local fiable

Base locale

- produits
- prix
- stock local
- tickets

Vente Offline

1. Lecture locale
2. Enregistrement ticket SQLite
3. Émission événement différé

Sync Engine

- Push dès Internet
- Résolution par horodatage + priorité serveur

---

7. Module Vente B2B (Back-office)

Stack

- Next.js (UI)
- NestJS (API)

Workflow

Devis → Proforma → BL → Facture

Chaque étape = **changement d'état**, jamais suppression.

---

8. Sécurité, RBAC & Audit

RBAC (NestJS Guards)

- RoleGuard
- PermissionGuard

Audit

Table `audit_logs`

- user
- action
- before
- after
- timestamp

Pourquoi ?

➡️ Obligatoire pour caisse & finance.

---

9. Sync & Kafka (Cœur du Offline)

Événements

- SaleCompleted
- StockMoved
- PriceUpdated

Avantage Kafka

- Rejouable
- Découplé
- Résilient

---

10. Déploiement & Scalabilité

- Docker par service
- Kubernetes (namespace par client)
- PostgreSQL central
- SQLite local

---

11. Roadmap Technique

Phase 1

- Catalogue
- Stock
- POS

Phase 2

- Import avancé
- Sync robuste

Phase 3

- Mobile dirigeant
- IA prévision stock

Conclusion

Ce découpage te permet :

- de développer **module par module**
- sans dette technique
- avec une vraie logique ERP pro